

发生在它们第一次变成错误时。另一方面, 值 `src1[i1]` 和 `src2[i2]` 之间的比较(第 6 行), 对于通常的数据来说, 都是非常难以预测的。这个比较控制一个条件分支, 运行在随机数据上时, 得到的 CPE 大约为 15.0(这里元素的数量为 $2n$)。

重写这段代码, 使得可以用一个条件传送语句来实现第一个循环中条件语句(第 6~9 行)的功能。

5.12 理解内存性能

到目前为止我们写的所有代码, 以及运行的所有测试, 只访问相对比较少量的内存。例如, 我们都是在长度小于 1000 个元素的向量上测试这些合并函数, 数据量不会超过 8000 个字节。所有的现代处理器都包含一个或多个高速缓存(cache)存储器, 以对这样少量的存储器提供快速的访问。本节会进一步研究涉及加载(从内存读到寄存器)和存储(从寄存器写到内存)操作的程序的性能, 只考虑所有的数据都存放在高速缓存中的情况。在第 6 章, 我们会更详细地探究高速缓存是如何工作的, 它们的性能特性, 以及如何编写充分利用高速缓存的代码。

如图 5-11 所示, 现代处理器有专门的功能单元来执行加载和存储操作, 这些单元有内部的缓冲区来保存未完成的内存操作请求集合。例如, 我们的参考机有两个加载单元, 每一个可以保存多达 72 个未完成的读请求。它还有一个存储单元, 其存储缓冲区能保存最多 42 个写请求。每个这样的单元通常可以每个时钟周期开始一个操作。

5.12.1 加载的性能

一个包含加载操作的程序的性能既依赖于流水线的的能力, 也依赖于加载单元的延迟。在参考机上运行合并操作的实验中, 我们看到除了使用 SIMD 操作时以外, 对任何数据类型组合和合并操作来说, CPE 从没有到过 0.50 以下。一个制约示例的 CPE 的因素是, 对于每个被计算的元素, 所有的示例都需要从内存读一个值。对两个加载单元而言, 其每个时钟周期只能启动一条加载操作, 所以 CPE 不可能小于 0.50。对于每个被计算的元素必须加载 k 个值的应用, 我们不可能获得低于 $k/2$ 的 CPE(例如参见家庭作业 5.15)。

到目前为止, 我们在示例中还没有看到加载操作的延迟产生的影响。加载操作的地址只依赖于循环索引 i , 所以加载操作不会成为限制性能的关键路径的一部分。

要确定一台机器上加载操作的延迟, 我们可以建立由一系列加载操作组成的一个计算, 一条加载操作的结果决定下一条操作的地址。作为一个例子, 考虑函数图 5-31 中的函数 `list_len`, 它计算一个链表的长度。在这个函数的循环中, 变量 `ls` 的每个后续值依赖于指针引用 `ls->next` 读出的值。测试表明函数 `list_len` 的 CPE 为 4.00, 我们认为这直接表明了加载操作的延迟。要弄懂这一点, 考虑循环的汇编代码:

```

Inner loop of list_len
ls in %rdi, len in %rax
1  .L3:                                loop:
2  addq    $1, %rax                    Increment len
3  movq    (%rdi), %rdi               ls = ls->next

```

```

1  typedef struct ELE {
2      struct ELE *next;
3      long data;
4  } list_ele, *list_ptr;
5
6  long list_len(list_ptr ls) {
7      long len = 0;
8      while (ls) {
9          len++;
10         ls = ls->next;
11     }
12     return len;
13 }

```

图 5-31 链表函数。其性能受限于加载操作的延迟


```

4      testq    %rdi, %rdi      Test 1s
5      jne      .L3             If nonnull, goto loop

```

第3行上的 `movq` 指令是这个循环中关键的瓶颈。后面寄存器 `%rdi` 的每个值都依赖于加载操作的结果，而加载操作又以 `%rdi` 中的值作为它的地址。因此，直到前一次迭代的加载操作完成，下一次迭代的加载操作才能开始。这个函数的 CPE 等于 4.00，是由加载操作的延迟决定的。事实上，这个测试结果与文档中参考机的 L1 级 cache 的 4 周期访问时间是一致的，相关内容将在 6.4 节中讨论。

5.12.2 存储的性能

在迄今为止所有的示例中，我们只分析了大部分内存引用都是加载操作的函数，也就是从内存位置读到寄存器中。与之对应的是存储(store)操作，它将一个寄存器值写到内存。这个操作的性能，尤其是与加载操作的相互关系，包括一些很细微的问题。

与加载操作一样，在大多数情况中，存储操作能够在完全流水线化的模式中工作，每个周期开始一条新的存储。例如，考虑图 5-32 中所示的函数，它们将一个长度为 n 的数组 `dest` 的元素设置为 0。我们测试结果为 CPE 等于 1.00。对于只具有单个存储功能单元的机器，这已经达到了最佳情况。

```

1  /* Set elements of array to 0 */
2  void clear_array(long *dest, long n) {
3      long i;
4      for (i = 0; i < n; i++)
5          dest[i] = 0;
6  }

```

图 5-32 将数组元素设置为 0 的函数。该代码 CPE 达到 1.0

与到目前为止我们已经考虑过的其他操作不同，存储操作并不影响任何寄存器值。因此，就其本性来说，一系列存储操作不会产生数据相关。只有加载操作会受存储操作结果的影响，因为只有加载操作能从由存储操作写的那个位置读回值。图 5-33 所示的函数 `write_read` 说明了加载和存储操作之间可能的相互影响。这幅图也展示了该函数的两个示例执行，是对两元素数组 `a` 调用的，该数组的初始内容为 -10 和 17，参数 `cnt` 等于 3。这些执行说明了加载和存储操作的一些细微之处。

在图 5-33 的示例 A 中，参数 `src` 是一个指向数组元素 `a[0]` 的指针，而 `dest` 是一个指向数组元素 `a[1]` 的指针。在此种情况中，指针引用 `*src` 的每次加载都会得到值 -10。因此，在两次迭代之后，数组元素就会分别保持固定为 -10 和 -9。从 `src` 读出的结果不受对 `dest` 的写的影响。在较大次数的迭代上测试这个示例得到 CPE 等于 1.3。

在图 5-33 的示例 B 中，参数 `src` 和 `dest` 都是指向数组元素 `a[0]` 的指针。在这种情况下，指针引用 `*src` 的每次加载都会得到指针引用 `*dest` 的前次执行存储的值。因而，一系列不断增加的值会被存储在这个位置。通常，如果调用函数 `write_read` 时参数 `src` 和 `dest` 指向同一个内存位置，而参数 `cnt` 的值为 $n > 0$ ，那么净效果是将这个位置设置为 $n-1$ 。这个示例说明了一个现象，我们称之为写/读相关(write/read dependency)——一个内存读的结果依赖于一个最近的内存写。我们的性能测试表明示例 B 的 CPE 为 7.3。写/读相关导致处理速度下降约 6 个时钟周期。

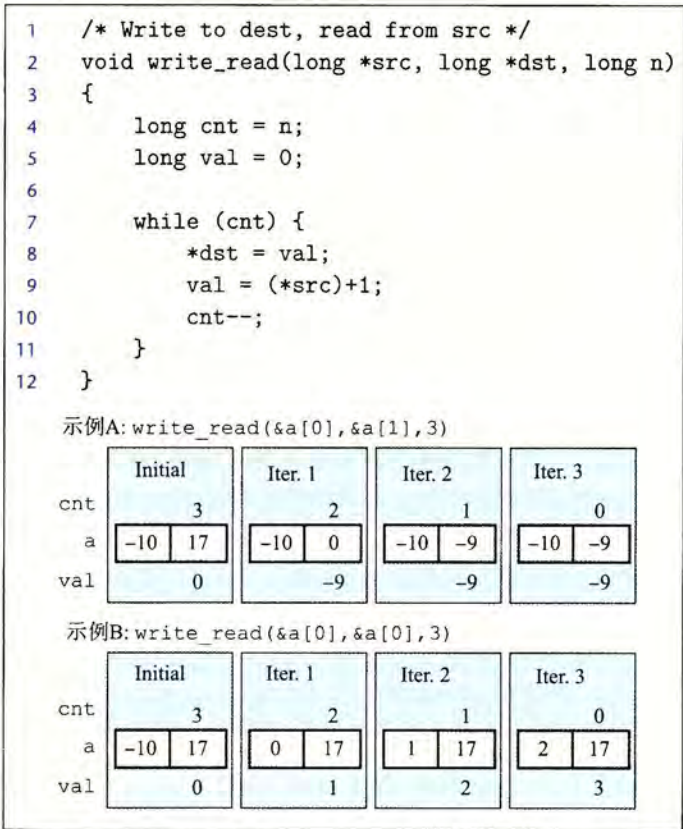


图 5-33 写和读内存位置的代码，以及示例执行。这个函数突出的是当参数 src 和 dest 相等时，存储和加载之间的相互影响

为了了解处理器如何区别这两种情况，以及为什么一种情况比另一种运行得慢，我们必须更加仔细地看看加载和存储执行单元，如图 5-34 所示。存储单元包含一个存储缓冲区，它包含已经被发射到存储单元而又还没有完成的存储操作的地址和数据，这里的完成包括更新数据高速缓存。提供这样一个缓冲区，使得一系列存储操作不必等待每个操作都更新高速缓存就能够执行。当一个加载操作发生时，它必须检查存储缓冲区中的条目，看有没有地址相匹配。如果有地址相匹配（意味着在写的字节与在读的字节有相同的地址），它就取出相应的数据条目作为加载操作的结果。

GCC 生成的 write_read 内循环代码如下：

```
Inner loop of write_read
src in %rdi, dst in %rsi, val in %rax
.L3:      loop:
movq     %rax, (%rsi)    Write val to dst
movq     (%rdi), %rax    t = *src
addq     $1, %rax        val = t+1
subq     $1, %rdx        cnt--
jne      .L3             If != 0, goto loop
```

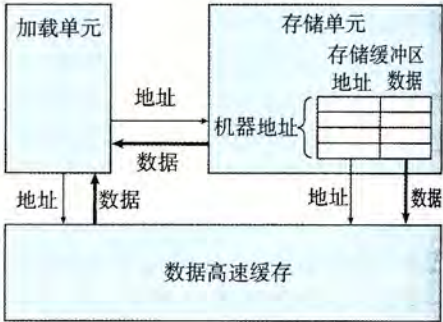


图 5-34 加载和存储单元的细节。存储单元包含一个未执行的写的缓冲区。加载单元必须检查它的地址是否与存储单元中的地址相符，以发现写/读相关

图 5-35 给出了这个循环代码的数据流表示。指令 movq %rax, (%rsi) 被翻译成两个操作：s_

addr 指令计算存储操作的地址，在存储缓冲区创建一个条目，并且设置该条目的地址字段。s_data 操作设置该条目的数据字段。正如我们会看到的，两个计算是独立执行的，这对程序的性能来说很重要。这使得参考机中不同的功能单元来执行这些操作。

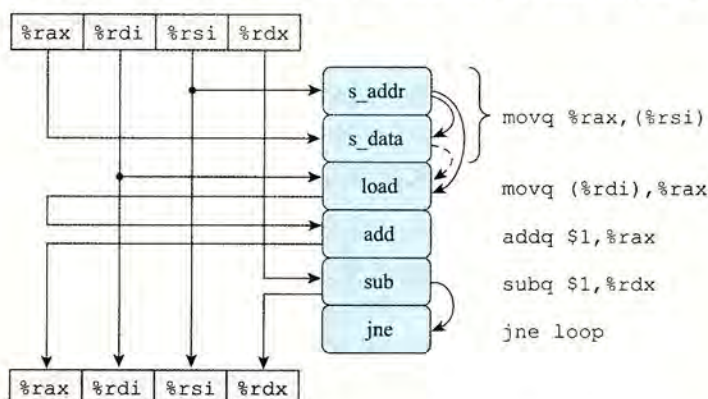


图 5-35 write_read 内循环代码的图形化表示。第一个 movl 指令被译码两个独立的操作，计算存储地址和将数据存储到内存

除了由于写和读寄存器造成的操作之间的数据相关，操作符右边的弧线表示这些操作隐含的相关。特别地，s_addr 操作的地址计算必须在 s_data 操作之前。此外，对指令 movq (%rdi), %rax 译码得到的 load 操作必须检查所有未完成的存储操作的地址，在这个操作和 s_addr 操作之间创建一个数据相关。这张图中 s_data 和 load 操作之间有虚弧线。这个数据相关是有条件的：如果两个地址相同，load 操作必须等待直到 s_data 将它的结果存放到存储缓冲区中，但是如果两个地址不同，两个操作就可以独立地进行。

图 5-36 说明了 write_read 内循环操作之间的数据相关。在图 5-36a 中，重新排列了操作，让相关显得更清楚。我们标出了三个涉及加载和存储操作的相关，希望引起大家特别的注意。标号为(1)的弧线表示存储地址必须在数据被存储之前计算出来。标号为(2)的弧线表示需要 load 操作将它的地址与所有未完成的存储操作的地址进行比较。最后，标号为(3)的虚弧线表示条件数据相关，当加载和存储地址相同时会出现。

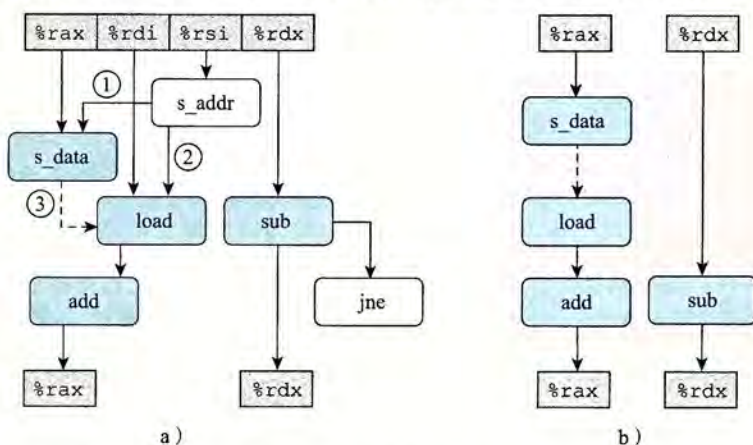


图 5-36 抽象 write_read 的操作。我们首先重新排列图 5-35 的操作(a)，然后只显示那些使用一次迭代中的值为下一次迭代产生新值的操作(b)

图 5-36b 说明了当移走那些不直接影响迭代与迭代之间数据流的操作之后，会发生什么。这个数据流图给出两个相关链：左边的一条，存储、加载和增加数据值（只对地址相同的情况有效），右边的一条，减小变量 cnt。

现在我们可以理解函数 `write_read` 的性能特征了。图 5-37 说明的是内循环的多次迭代形成的数据相关。对于图 5-33 示例 A 的情况，有不同的源和目的地址，加载和存储操作可以独立进行，因此唯一的关键路径是由减少变量 `cnt` 形成的，这使得 CPE 等于 1.0。对于图 5-33 示例 B 的情况，源地址和目的地址相同，`s_data` 和 `load` 指令之间的数据相关使得关键路径的形成包括了存储、加载和增加数据。我们发现顺序执行这三个操作一共需要 7 个时钟周期。

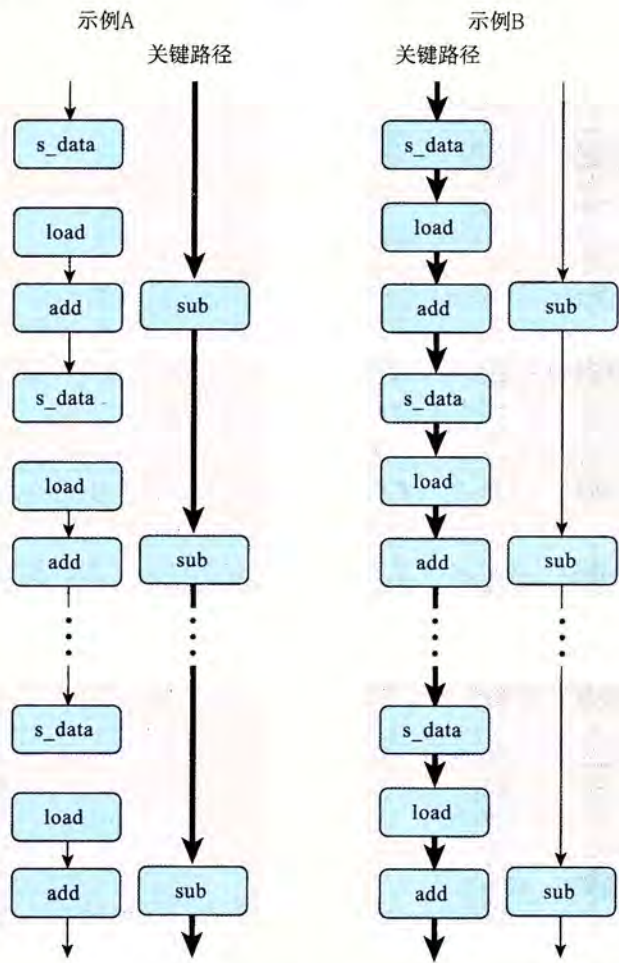


图 5-37 函数 `write_read` 的数据流表示。当两个地址不同时，唯一的关键路径是减少 `cnt` (示例 A)。当两个地址相同时，存储、加载和增加数据的链形成了关键路径 (示例 B)

这两个例子说明，内存操作的实现包括许多细微之处。对于寄存器操作，在指令被译码成操作的时候，处理器就可以确定哪些指令会影响其他哪些指令。另一方面，对于内存操作，只有到计算出加载和存储的地址被计算出来以后，处理器才能确定哪些指令会影响其他的哪些。高效地处理内存操作对许多程序的性能来说至关重要。内存子系统使用了很多优化，例如当操作可以独立地进行时，就利用这种潜在的并行性。

 **练习题 5.10** 作为另一个具有潜在的加载-存储相互影响的代码，考虑下面的函数，它将一个数组的内容复制到另一个数组：

```
1 void copy_array(long *src, long *dest, long n)
2 {
3     long i;
4     for (i = 0; i < n; i++)
5         dest[i] = src[i];
6 }
```

假设 *a* 是一个长度为 1000 的数组，被初始化为每个元素 *a*[*i*] 等于 *i*。

- 调用 `copy_array(a+1,a,999)` 的效果是什么？
- 调用 `copy_array(a,a+1,999)` 的效果是什么？
- 我们的性能测试表明问题 A 调用的 CPE 为 1.2(循环展开因子为 4 时，该值下降到 1.0)，而问题 B 调用的 CPE 为 5.0。你认为是什么因素造成了这样的性能差异？
- 你预计调用 `copy_array(a,a,999)` 的性能会是怎样的？

 **练习题 5.11** 我们测量出前置和函数 `psum1`(图 5-1) 的 CPE 为 9.00，在测试机器上，要执行的基本操作——浮点加法的延迟只是 3 个时钟周期。试着理解为什么我们的函数执行效果这么差。

下面是这个函数内循环的汇编代码：

Inner loop of psum1

a in %rdi, i in %rax, cnt in %rdx

1	.L5:	loop:
2	<code>vmovss -4(%rsi,%rax,4), %xmm0</code>	<i>Get p[i-1]</i>
3	<code>vaddss (%rdi,%rax,4), %xmm0, %xmm0</code>	<i>Add a[i]</i>
4	<code>vmovss %xmm0, (%rsi,%rax,4)</code>	<i>Store at p[i]</i>
5	<code>addq \$1, %rax</code>	<i>Increment i</i>
6	<code>cmpq %rdx, %rax</code>	<i>Compare i:cnt</i>
7	<code>jne .L5</code>	<i>If !=, goto loop</i>

参考对 `combine3`(图 5-14)和 `write_read`(图 5-36)的分析，画出这个循环生成的数据相关图，再画出计算进行时由此形成的关键路径。解释为什么 CPE 如此之高。

 **练习题 5.12** 重写 `psum1`(图 5-1)的代码，使之不需要反复地从内存中读取 *p*[*i*] 的值。不需要使用循环展开。得到的代码测试出的 CPE 等于 3.00，受浮点加法延迟的限制。

5.13 应用：性能提高技术

虽然只考虑了有限的一组应用程序，但是我们能得出关于如何编写高效代码的很重要的经验教训。我们已经描述了许多优化程序性能的基本策略：

1) 高级设计。为遇到的问题选择适当的算法和数据结构。要特别警觉，避免使用那些会渐进地产生糟糕性能的算法或编码技术。

2) 基本编码原则。避免限制优化的因素，这样编译器就能产生高效的代码。

- 消除连续的函数调用。在可能时，将计算移到循环外。考虑有选择地妥协程序的模块性以获得更大的效率。
- 消除不必要的内存引用。引入临时变量来保存中间结果。只有在最后的值计算出来时，才将结果存放到数组或全局变量中。